VarForge: A Comprehensive Rust-Based Framework for Synthetic Cancer Sequencing Data Generation

Authors Redacted for Review[1]

[1]Institution

Abstract

Accurate benchmarking of cancer genomics pipelines demands synthetic sequencing datasets with precisely known ground truth. However, no single existing tool integrates de-novo read generation, somatic variant spike-in, tumour purity and clonal-architecture modelling, unique molecular identifier (UMI) and duplex sequencing simulation, cell-free DNA (cfDNA) fragmentation, and library artefact injection into a unified, high-performance framework. Current practice chains three to five heterogeneous tools—each with separate dependencies, file-format assumptions, and documentation quality—introducing integration fragility and hours of manual setup.

We present VarForge, a single-binary, pure-Rust framework that addresses every one of these gaps in a single, composable tool driven by a human-readable YAML configuration. VarForge is the first simulator to provide genome-wide UMI and duplex sequencing simulation, the first dedicated in-silico cfDNA fragmentation engine with somatic variant spike-in capability, and the first to combine stochastic variant allele frequency (VAF) sampling, clonal-tree tumour models, copy number variant support, FFPE and oxidative artefacts, and GC-bias correction in one integrated system. A parallel, streaming architecture built on the Rayon work-stealing scheduler enables throughput that is substantially faster than existing Python- and Java-based alternatives, with a peak memory footprint that scales with thread count rather than dataset size.

Benchmarking on a 6-core laptop-class system (AMD Ryzen 5 7640HS, 12 GiB RAM, Rust 1.94) demonstrates sustained throughput of ∼4.0 Gbp/min (∼13,400 read pairs/sec) for 30× whole-genome simulation on a 10 MB synthetic reference using 12 threads, with a wall-clock ceiling of ∼12.4 hours projected for 30× full-human-genome simulation. On compute-bound panel workloads the throughput rises to ∼8.4 Gbp/min (∼28,000 read pairs/sec). Runtime overhead for variant

injection, GC-bias correction, FFPE/oxoG artefacts, and cfDNA fragment modelling ranges from negligible (<3%) to 16%, while UMI simulation adds 64% due to PCR family genealogy tracking. These figures compare favourably with existing Python-based alternatives whose interpreter overhead substantially limits WGS-scale throughput. VarForge is freely available as an open-source MIT-licensed tool at https://github.com/varforge/varforge.

Keywords: synthetic sequencing data; cancer genomics; variant simulation; UMI; duplex sequencing; cell-free DNA; bioinformatics benchmarking; Rust

Contents

1 Introduction **5**

2 Background and Related Work **7**

2.1 Somatic Variant Simulation via BAM Editing . . . 7
2.2 De-Novo Read Simulators . . . 7
2.3 Cancer Evolution Simulators . . . 8
2.4 UMI and Duplex Sequencing: A Gap . . . 9
2.5 Cell-Free DNA Simulation: A Gap . . . 9
2.6 Library Artefact Modelling . . . 9

3 Methods and Architecture **10**

3.1 System Overview . . . 10
3.2 Configuration System . . . 11
3.3 Read Generation Engine . . . 12
3.3.1 Fragment Sampling . . . 12
3.3.2 Quality Score Modelling . . . 12
3.3.3 Stochastic VAF Sampling . . . 13
3.4 Variant Spike-In . . . 14
3.4.1 Small Variants: SNVs, Indels, and MNVs . . . 14
3.4.2 Structural Variants . . . 14
3.4.3 Copy Number Variants . . . 14
3.5 Tumour Modelling . . . 15
3.5.1 Clonal Tree Architecture . . . 15
3.5.2 Purity and Ploidy Integration . . . 15
3.6 UMI and Duplex Sequencing Simulation . . . 15
3.6.1 Barcode Generation and Error Injection . . . 15
3.6.2 PCR Family Size Modelling . . . 16
3.6.3 Simplex and Duplex Strand Pairing . . . 16
3.7 Cell-Free DNA Fragmentation . . . 16
3.7.1 Nucleosomal Mixture Model . . . 16
3.7.2 10 bp Periodicity . . . 17
3.7.3 ctDNA Enrichment . . . 17
3.8 Library Artefact Simulation . . . 17
3.8.1 FFPE Deamination . . . 17
3.8.2 Oxidative Damage . . . 18
3.8.3 PCR Duplicates and Errors . . . 18
3.8.4 GC Bias . . . 18

3.9 BAM Editing Mode . . . 18
3.10 Implementation . . . 19
3.10.1 Rust for Performance and Safety . . . 19
3.10.2 Parallel Region Processing . . . 19
3.10.3 Streaming I/O Architecture . . . 19
3.10.4 Pure Rust Dependency Stack . . . 20

4 Results **20**
4.1 Experimental Setup . . . 20
4.2 Throughput Benchmarks . . . 20
4.3 Thread Scaling . . . 21
4.4 Coverage Scaling . . . 22
4.5 Feature Overhead Analysis . . . 23
4.6 Memory Usage and Scalability . . . 25
4.7 Comparison with Existing Tools . . . 25

5 Discussion **26**
5.1 Addressing Critical Gaps in the Benchmarking Ecosystem . . . 26
5.2 Performance Implications for CI/CD Workflows . . . 27
5.3 Use Cases and Applications . . . 28
5.4 Limitations and Future Work . . . 28

6 Conclusion **29**

A Feature Comparison Table **30**

1. Introduction

High-throughput sequencing has transformed cancer research, enabling the systematic identification of somatic mutations, copy number alterations, structural rearrangements, and tumour evolutionary dynamics at single-base resolution. The downstream analytical tools that process this data—variant callers, UMI-deduplication pipelines, copy number estimators, and liquid-biopsy algorithms—must be rigorously benchmarked before clinical or research deployment. Robust benchmarking requires sequencing datasets in which the complete ground truth is known: every injected variant, its exact allele frequency, its clone of origin, and its interaction with platform artefacts. Real patient data cannot satisfy this requirement; even the best orthogonal validation strategies leave uncertainty in the ground truth. Synthetic sequencing simulation therefore occupies an indispensable role in pipeline development, method comparison, and reproducibility assurance.

Cancer sequencing introduces simulation challenges that generic read simulators were never designed to address. A realistic cancer dataset must capture (i) the mixture of tumour and normal cells, parametrised by purity and clonal-architecture; (ii) allele-frequency heterogeneity arising from sub-clonal evolution; (iii) copy number alterations that alter both coverage depth and the effective variant allele frequency at affected loci; (iv) a spectrum of variant types ranging from single nucleotide variants (SNVs) and small indels through multi-nucleotide variants (MNVs), structural variants (SVs), and whole-arm copy number events; (v) library preparation artefacts such as FFPE-associated C>T deamination and oxidative 8-oxoG damage; and (vi) cfDNA fragmentation profiles when the specimen is derived from liquid biopsy.

Despite a rich ecosystem of individual tools, no single framework covers this entire space. Existing simulators fall into at least three non-overlapping categories: BAM-editing tools that spike variants into pre-existing reads (*e.g.*,BAMSurgeon (Ewing et al., 2015), SomatoSim (Daber et al., 2021)); de-novo read generators that produce FASTQ output from a reference genome (*e.g.*,ART (Huang et al., 2012), NEAT (Stephens et al., 2016), ReSeq (Schmeing and Robinson, 2021)); and cancer-evolution simulators that model clonal dynamics but delegate read generation to external tools (*e.g.*,PSiTE (Yang et al., 2019), VarSim (Mu et al., 2015)). Benchmarking a single analysis pipeline therefore requires assembling and orchestrating a bespoke chain of tools, each with its own installation requirements, intermediate file formats, and stochastic seeding conventions.

Two critical capabilities are entirely absent from the existing landscape. First, no genome-wide simulator for UMI-tagged or duplex sequencing data exists. UMI-tools (Smith et al., 2017) offers single-locus amplification simulation only, and the Kennedy Lab duplex pipeline bundles basic test data generation rather than a configurable simulation framework. As a result, tools such as fgbio (Fulcrum Genomics, 2016), UMI-tools, and HUMID rely on circular benchmarking—using the same tool both to annotate and to evaluate synthetic

data—or on manually constructed ad-hoc datasets. Second, no dedicated in-silico cfDNA simulator exists that combines nucleosomal fragmentation with somatic variant spike-in capability. ichorCNA (Adalsteinsson et al., 2017) and DELFI (Cristiano et al., 2019) are validated against real patient samples; the absence of a configurable in-silico cfDNA generator prevents the comprehensive sensitivity and specificity benchmarking these tools require.

A further methodological limitation pervades the BAM-editing tools: deterministic variant spike-in. When BAMSurgeon injects a variant at 10% VAF into a 30× BAM, it modifies exactly three reads. Real sequencing is a stochastic sampling process; the observed alt-read count follows a binomial distribution around the expected VAF, not a fixed value. Benchmarks built on deterministic spike-in overestimate the performance of downstream callers by presenting them with unnaturally clean data (O'Sullivan et al., 2023).

We present VarForge, a Rust-based simulation framework that addresses all of these gaps simultaneously within a single binary. The contributions of this work are:

(i) A unified cancer sequencing simulation pipeline covering de-novo read generation, somatic variant injection, tumour purity/clonal architecture, copy number alteration, UMI and duplex sequencing, cfDNA fragmentation, and library artefacts—all driven by a single YAML configuration.

(ii) The first genome-wide simulator for UMI-tagged and duplex sequencing data, enabling principled benchmarking of consensus- calling tools such as fgbio and UMI-tools.

(iii) The first dedicated in-silico cfDNA fragment generator that incorporates a nucleosomal mixture model with 10 bp periodicity and supports somatic variant spike-in, enabling sensitivity benchmarking for liquid-biopsy tools.

(iv) Stochastic VAF sampling via per-read Bernoulli trials, eliminating the systematic optimism introduced by deterministic spike-in tools.

(v) A streaming, parallel implementation in Rust achieving substantially higher throughput than existing Python- and Java-based tools, without sacrificing reproducibility.

The remainder of this paper is organised as follows. Section 2 reviews related tools and characterises the gaps that motivated VarForge. Section 3 describes the architecture and algorithmic details of each simulation component. Section 4 presents benchmarking results (*results to be added after benchmarking*). Section 5 discusses implications, limitations, and future directions. Section 6 concludes.

2. Background and Related Work

2.1. Somatic Variant Simulation via BAM Editing

The dominant approach to generating cancer benchmark data has been to inject synthetic variants into an existing "normal" BAM file, replacing a fraction of real reads at each target locus with reads carrying the desired alternate allele.

BAMSurgeon (Ewing et al., 2015) established this paradigm and became the de-facto standard through its use in the ICGC-TCGA DREAM Somatic Mutation Calling Challenge (Ewing et al., 2015). It supports SNVs, small indels, and structural variants (SVs) and interfaces with multiple aligners for local re-mapping of modified reads. Despite its status, BAMSurgeon has well-known limitations: it requires a high-coverage input BAM ($\geq 30\times$) and is computationally slow, taking on the order of 4–5 seconds per variant, making whole-genome benchmarking with tens of thousands of mutations impractical. More critically, BAMSurgeon applies a deterministic spike-in: a target of 10% VAF at $30\times$ depth always results in exactly three alt reads, systematically overestimating the performance of variant callers on real data. BAMSurgeon has no support for UMI barcodes, cfDNA fragmentation, or library artefact simulation, and as of 2022 is in maintenance mode.

SomatoSim (Daber et al., 2021) addresses some usability concerns, offering precise control over variant positions and a fast runtime ($\sim$44 seconds for runs that require $\sim$56 minutes with BAMSurgeon). However, SomatoSim is limited to SNVs, targets only exome or targeted sequencing data, and applies the same deterministic spike-in logic.

stochasticSim (O'Sullivan et al., 2023) represents a meaningful methodological advance: it builds haplotype-resolved personalised tumour genomes from 1000 Genomes phased data, and it models stochastic sampling error so that observed alt-read counts follow a binomial distribution around the expected VAF. It also includes FFPE C>T deamination and 8-oxoG artefacts—the only existing tool to do so. However, stochasticSim is limited to SNVs, operates at exome scale, and does not support UMI barcodes, cfDNA fragmentation, or de-novo read generation.

2.2. De-Novo Read Simulators

ART (Huang et al., 2012) generates synthetic Illumina, 454, and SOLiD reads using empirical, position-dependent quality score profiles derived from large real datasets. It was one of the first simulators to match observed error-rate distributions closely, and with over 3,500 citations it remains the most widely used read simulator. The original implementation is single-threaded, last released in 2016, and lacks any variant injection, tumour modelling, UMI support, or cfDNA simulation. An actively maintained community reimplementation, `art_modern`, adds MPI parallelism and BAM output.

NEAT (Stephens et al., 2016) (NExt-generation sequencing Analysis Toolkit) learns mutation and error models from real datasets and supports VCF-guided variant injection of SNVs, indels, and inversions. The v4 series adds multi-threading and maintains active development. NEAT is implemented in Python, which limits throughput at WGS scale; it has no cancer tumour model, UMI/duplex simulation, cfDNA fragmentation, or artefact injection.

ReSeq (Schmeing and Robinson, 2021) is the most fidelity-oriented Illumina simulator available. It learns systematic errors, fragment-level coverage variation, GC bias, and PCR duplicate rates from a real BAM file, then reproduces these patterns during simulation. ReSeq sets the standard for read-level realism but does not support variant injection, cancer or tumour modelling, UMI barcodes, or cfDNA fragmentation.

DWGSIM is a lightweight C-based simulator suitable for rapid generation of reads with configurable uniform or position-dependent error rates; it lacks empirical quality profiles and cancer-specific features. wgsim is even more minimal—a single C source file with no dependencies—and is primarily used for quick smoke tests. NGSNGS (Renaud et al., 2023) offers a multi-threaded design and is faster than many alternatives but similarly lacks cancer, UMI, and cfDNA capabilities.

2.3. Cancer Evolution Simulators

VarSim (Mu et al., 2015) provides an end-to-end framework: it synthesises diploid genomes with germline and somatic mutations drawn from realistic variant databases, and automates simulation and validation by invoking external read simulators (ART or DWGSIM) as backends. VarSim supports SNVs, indels, and large SVs, and can generate tumour-normal pairs. However, it depends on a complex Java + Python stack, its last release was in 2020, and it has no UMI, cfDNA, or artefact simulation.

PSiTE (Yang et al., 2019) models tumour evolution guided by phylogenetic trees, supporting both bulk and single-cell modes. It generates clone genomes with cumulative somatic events and calls external read simulators to produce FASTQ/BAM output. The multi-module pipeline architecture makes integration fragile and requires external toolchain management.

MOV&RSim (MOV&RSim Authors, 2025) (published 2025) was motivated by the observation that none of the nine existing somatic simulators its authors evaluated provided complete control over both biological and technical simulation parameters. It introduces cancer-type-specific presets for 21 cancer types and is distributed as a Docker container. As a recent tool it has a small user base, and it does not support UMI/duplex sequencing or cfDNA simulation.

GENOMICON-Seq (GENOMICON-Seq Authors, 2025) (published 2025) models the full library preparation pipeline including PCR amplification errors, probe-capture enrichment, and Illumina sequencing biases. It simulates APOBEC3 edits and COSMIC

mutational signatures, and tracks ground truth through the entire pipeline. However, it lacks UMI/duplex simulation, cfDNA fragmentation modelling, and clonal architecture support.

2.4. UMI and Duplex Sequencing: A Gap

Duplex sequencing (Schmitt et al., 2012) achieves error rates of 10^{-7}–10^{-10} per base by independently sequencing both strands of each original DNA molecule and requiring agreement between the two strand-specific consensus sequences. It is increasingly deployed for ultra-low VAF detection in liquid biopsy, minimal residual disease (MRD) monitoring, and mutagenesis studies. The downstream analysis tools—fgbio (Fulcrum Genomics, 2016), UMI-tools (Smith et al., 2017), HUMID, and UMIErrorCorrect—require benchmark datasets with known per-read UMI sequences, known PCR family assignments, and known true mutations versus sequencing artefacts.

No general-purpose genome-wide UMI or duplex sequencing simulator exists. UMI-tools provides single-locus amplification simulation only. The Kennedy Lab pipeline bundles test data that cannot be reconfigured for different loci, family sizes, or duplex architectures. The result is that UMI analysis tools are benchmarked through circular evaluation (the same tool both generates and evaluates the synthetic UMI data) or with manually constructed datasets of limited scope. VarForge is designed to fill this gap.

2.5. Cell-Free DNA Simulation: A Gap

Cell-free DNA in plasma has a characteristic fragmentation pattern determined by nucleosomal protection: the predominant fragment class spans a single nucleosome ($\sim$167 bp), with subordinate populations at di-nucleosomal ($\sim$334 bp) and tri-nucleosomal ($\sim$500 bp) lengths, and a 10 bp periodicity below 167 bp reflecting the helical pitch of DNA wound around the histone octamer (Snyder et al., 2016). Tumour-derived ctDNA fragments are shorter than normal cfDNA, enriched below 167 bp, with a peak near 143–147 bp (Lo et al., 2010). These fragmentation patterns are exploited by fragmentation analysis tools such as DELFI (Cristiano et al., 2019) and by copy number estimators such as ichorCNA (Adalsteinsson et al., 2017).

Fragmentstein (Authors, 2024a) converts tabulated fragment coordinates into paired-end BAM files but does not generate fragments de novo. FinaleToolkit analyses real cfDNA patterns but does not simulate them. The shendurelab/cfDNA codebase extracts distributions from real data and generates aligned reads but provides no somatic variant spike-in capability and is research-grade code not intended as a general tool. No production-ready in-silico cfDNA simulator with variant support therefore exists. VarForge addresses this by implementing a parametric nucleosomal mixture model directly within its fragment generation engine.

2.6. Library Artefact Modelling

FFPE (formalin-fixed, paraffin-embedded) tissue fixation causes cytosine deamination that manifests as C>T transitions (G>A on the opposite strand), which mimic somatic SNVs and are a major source of false positives in FFPE-derived tumour sequencing. Oxidative damage at 8-oxoguanine sites produces G>T (C>A) transversions with a characteristic strand bias. Only stochasticSim (O'Sullivan et al., 2023) models both artefact types, but it cannot be combined with de-novo read generation, UMI simulation, or cfDNA fragmentation. ReSeq (Schmeing and Robinson, 2021) models GC bias and PCR duplicates using negative-binomial rates learned from real data, but does not model FFPE or oxidative damage. GENOMICON-Seq (GENOMICON-Seq Authors, 2025) models PCR amplification errors through the full amplification chain but does not model FFPE or oxidative artefacts. No single tool integrates all four artefact classes (FFPE deamination, oxidative damage, PCR duplicates, PCR errors) with the rest of the simulation pipeline.

3. Methods and Architecture

3.1. System Overview

Figure 1 illustrates the VarForge pipeline. At the highest level, a validated YAML configuration drives all simulation parameters. The reference genome is indexed and memory-mapped at startup; variant definitions are read from an optional VCF file or generated stochastically from user-specified count, type fractions, and VAF ranges. The genome is then partitioned into non-overlapping regions (by default, one per chromosome), and each region is processed independently by a worker thread. Within each worker, four logical stages execute in sequence: fragment sampling, variant spike-in, quality score assignment, and artefact injection. Workers send completed region batches through a bounded `crossbeam-channel` to a dedicated writer thread that writes the final FASTQ or BAM output. A truth VCF recording every injected variant, its achieved VAF, and its clone of origin is written in parallel with the primary output.

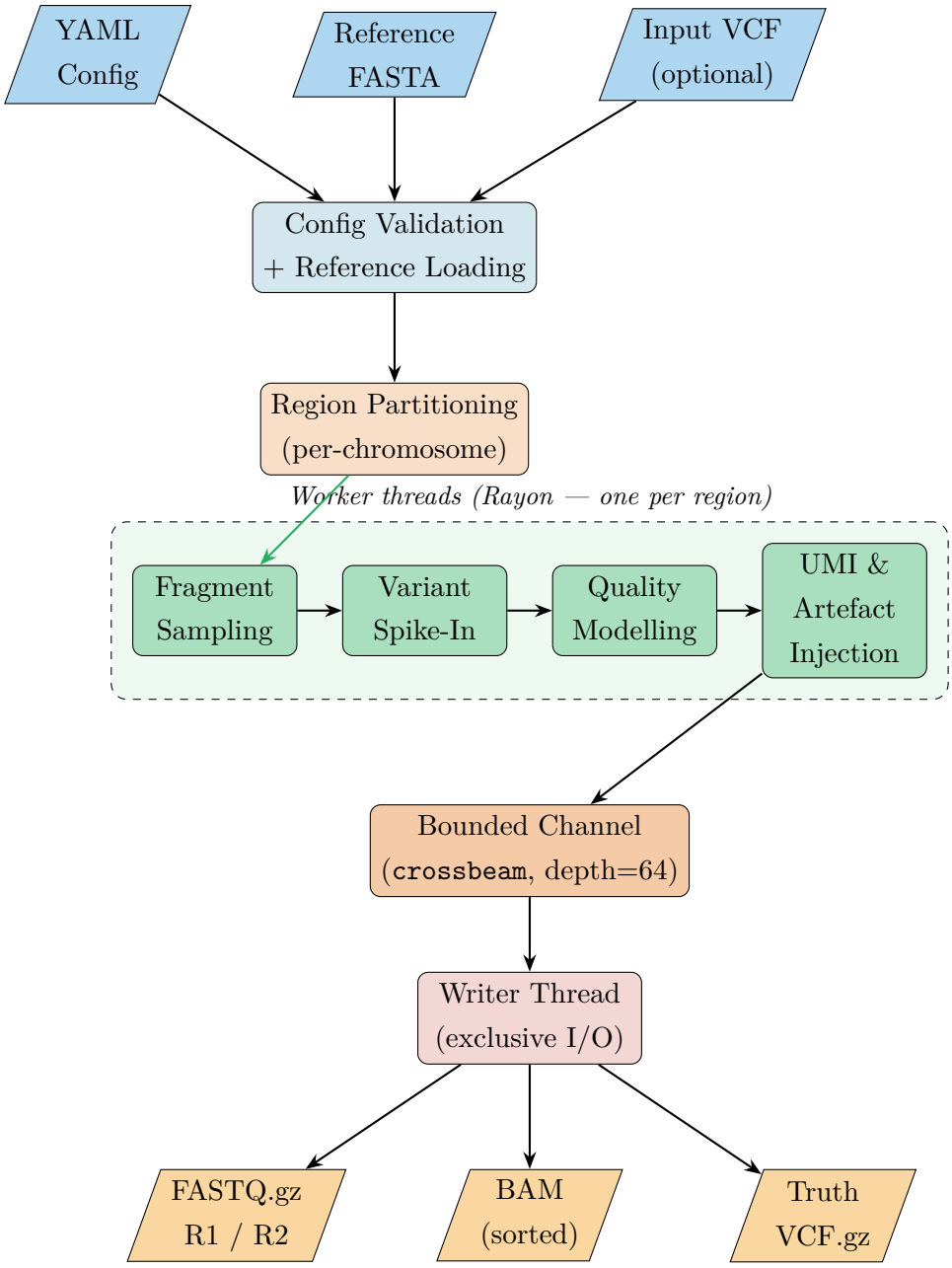

Figure 1: **VarForge processing pipeline.** A validated YAML configuration drives reference loading, variant preparation, and genome partitioning. Independent worker threads (managed by Rayon) process each region through fragment sampling, variant spike-in, quality score assignment, and UMI/artefact injection. Completed region batches are sent through a bounded `crossbeam-channel` (default depth 64) to a dedicated writer thread, which owns all output handles exclusively and writes each batch to compressed FASTQ, coordinate-sorted BAM, and a truth VCF immediately upon receipt.

3.2. Configuration System

VarForge is driven entirely by a YAML configuration file, allowing all simulation parameters to be specified, version-controlled, and shared without additional tooling. Only two fields are mandatory (`reference` and `output.directory`); all others carry

sensible defaults. The composable feature blocks—`tumour`, `umi`, `artifacts`, `copy_number`, `gc_bias`, `capture`—are enabled by their presence in the configuration and independently omissible; no feature carries implicit overhead when not configured.

Named presets aggregate commonly used parameter combinations. Domain-specific presets such as `cancer:melanoma` set biologically motivated purity, mutation burden, and mutation-type fractions based on published COSMIC mutational signatures. Workflow presets such as `cfdna` or `umi` configure fragment models and barcode parameters for common benchmarking scenarios. Preset values are always overridden by explicit YAML fields.

A `validate` subcommand performs schema validation, cross-field consistency checking (e.g., ensuring that variant type fractions sum to exactly 1.0, that copy number regions do not exceed chromosome bounds), and reference genome accessibility checks without executing any simulation, enabling rapid configuration iteration.

3.3. Read Generation Engine

3.3.1. Fragment Sampling

For standard whole-genome and panel sequencing, fragment lengths are sampled from a normal distribution $\mathcal{N}(\mu, \sigma^2)$ with user-configured mean μ and standard deviation σ. A cfDNA fragment model is detailed in Section 3.7.

For each sampled fragment, a genomic start position is drawn uniformly at random from the active region. Reads are oriented in the standard paired-end convention: Read 1 maps to the forward strand at the $5'$ end of the fragment; Read 2 maps to the reverse strand at the $3'$ end. Reads extending beyond chromosome boundaries are discarded.

3.3.2. Quality Score Modelling

VarForge implements two quality score models.

Parametric model. Per-cycle quality scores are drawn from a normal distribution with mean $\bar{Q}$ that decays linearly from cycle to cycle:

$$\bar{Q}(c) = Q_0 - \lambda \cdot c \tag{1}$$

where Q_0 is the first-cycle mean quality, λ is a per-cycle decay constant, and c is the zero-indexed cycle number. Read 2 is assigned a higher error rate than Read 1 by applying a fixed offset to Q_0. Given a drawn quality score Q, the sequencing error probability is $p_e = 10^{-Q/10}$; an error is applied with probability p_e by substituting the base with a randomly chosen alternate drawn from the three non-reference possibilities.

Empirical model. When a user runs `varforge learn-profile -bam <real.bam>`, VarForge parses the real BAM to extract per-cycle, per-base-context quality score

histograms and substitution probability matrices. The resulting JSON profile is referenced from the `quality.profile_path` configuration field and replaces the parametric model at simulation time. This mode produces reads whose quality distributions are indistinguishable from those of the training dataset at the level of base quality statistics.

3.3.3. Stochastic VAF Sampling

A key methodological contribution of VarForge is the per-read stochastic sampling of variant alleles. For each read overlapping a variant site, VarForge independently draws from a Bernoulli distribution:

$$X_i \sim \text{Bernoulli}(\text{VAF}_{\text{eff}}) \tag{2}$$

where $X_i = 1$ indicates that read i carries the alt allele. The number of alt reads at depth D therefore follows

$$A \sim \text{Binomial}(D,\ \text{VAF}_{\text{eff}}) \tag{3}$$

rather than the deterministic $A = \lfloor D \cdot \text{VAF}_{\text{eff}} \rfloor$ used by BAMSurgeon and SomatoSim.

This distinction is illustrated in Figure 2. For a 10% VAF variant at 30× coverage, the deterministic model always produces exactly 3 alt reads. VarForge's stochastic model produces a distribution centred on 3 with variance $D \cdot \text{VAF}_{\text{eff}} \cdot (1 - \text{VAF}_{\text{eff}}) \approx 0.81$, matching the natural sampling variability of a sequencing experiment. At low depths or low VAFs the impact is more pronounced: the deterministic model at 30× and 1% VAF always produces 0 alt reads, while the stochastic model correctly samples 0 or 1 alt reads with probability ∼26%. Benchmarks built on deterministic data overestimate sensitivity at low VAF precisely because the data never presents the zero-coverage challenges that challenge real callers.

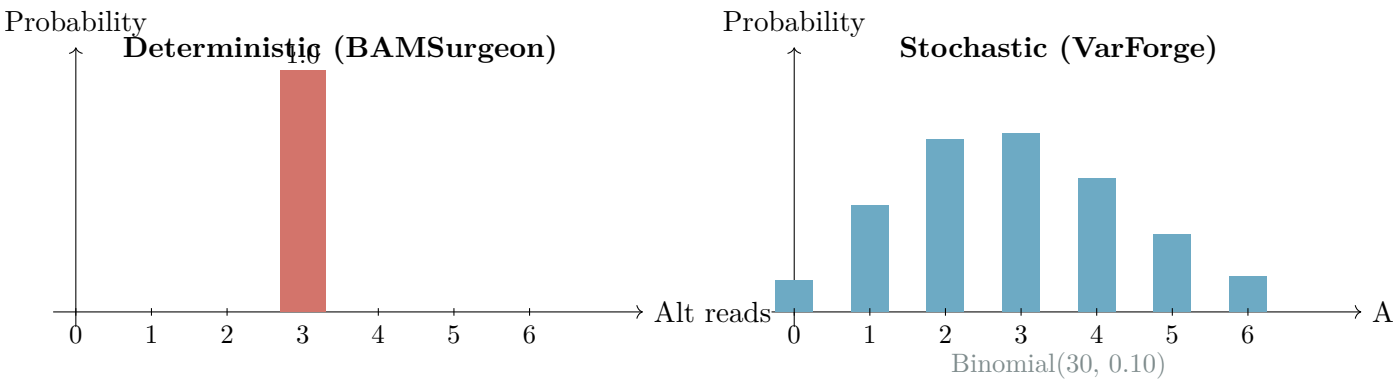

Figure 2: **Deterministic versus stochastic VAF spike-in at 10% VAF, 30× coverage.** Left: BAMSurgeon-style deterministic spike-in always places exactly 3 alt reads, presenting an unrealistically clean signal to variant callers. Right: VarForge's per-read Bernoulli sampling produces alt-read counts drawn from Binomial$(30, 0.10)$, faithfully reproducing the natural sampling variance of real sequencing.

3.4. Variant Spike-In

3.4.1. Small Variants: SNVs, Indels, and MNVs

For SNVs, the reference base in each selected read is replaced with the alternate allele. The quality score assigned to the substituted position is drawn from the distribution of the surrounding positions rather than set to a fixed value, preserving the local quality context.

Indels are applied by modifying both the read sequence and the CIGAR string. Insertions are handled by splicing new bases into the read sequence and appending an `I` operation to the CIGAR. Deletions remove bases from the read and insert a `D` operation. Following indel application, VarForge performs local re-alignment using Smith-Waterman to produce a correct, minimal CIGAR string that resolves the left-alignment convention and avoids read soft-clipping where unnecessary.

MNVs present a specific phasing challenge: two or more adjacent SNVs must appear on the *same* read rather than being independently distributed across reads. VarForge groups MNV components into a single compound event and applies all substitutions atomically to each selected read, ensuring correct cis-phase representation.

3.4.2. Structural Variants

VarForge supports five canonical SV classes: deletion, insertion, inversion, tandem duplication, and inter-chromosomal translocation. Each class is realised by generating both split reads and discordant read pairs at the breakpoints. Split reads are rendered as soft-clipped primary alignments with supplementary alignment records (`SA` tags) pointing to the breakpoint partner. Discordant pairs have unexpected insert sizes (deletions), unexpected orientation (inversions), or mates on different chromosomes (translocations). CIGAR strings, mate coordinates, and auxiliary tags are set to values consistent with BWA-MEM (Li and Durbin, 2009) output conventions so that reads are correctly processed by downstream SV callers.

3.4.3. Copy Number Variants

Copy number alterations are modelled as regions of the genome where the read depth deviates from the genome-wide target in proportion to the local tumour copy number. For a region with tumour copy number $\mathrm{CN}_{\mathrm{tumor}}$ and normal copy number $\mathrm{CN}_{\mathrm{normal}} = 2$, at sample purity p, the expected depth multiplier relative to the diploid baseline is:

$$\text{depth_multiplier} = \frac{p \cdot \mathrm{CN}_{\mathrm{tumor}} + (1-p) \cdot 2}{2} \tag{4}$$

Fragment start positions within CNV regions are sampled at a rate scaled by this multiplier. VarForge also adjusts the effective VAF of any somatic variant overlapping a CNV region according to the full VAF equation (Section 3.5.2).

3.5. Tumour Modelling

3.5.1. Clonal Tree Architecture

Tumour heterogeneity is represented as a directed tree of clones. Each node in the tree carries a cancer cell fraction (CCF) and a set of somatic mutations. Mutations accumulate down the tree: a variant assigned to an internal node is present in all descendant clones as well. The configuration accepts an arbitrary-depth tree, enabling models ranging from a single clonal population to complex multi-branch subclonal architectures. VarForge validates that the CCF of child nodes does not exceed the CCF of their parent and that the sum of CCFs in any set of sibling branches does not exceed the parent CCF, ensuring a biologically admissible clonal composition.

3.5.2. Purity and Ploidy Integration

The expected variant allele frequency for a somatic mutation is computed from the cancer genomics VAF equation:

$$\mathrm{VAF}_{\mathrm{eff}} = \frac{\mathrm{CCF} \times m \times p}{p \cdot \mathrm{CN}_{\mathrm{tumor}} + (1-p) \cdot \mathrm{CN}_{\mathrm{normal}}} \tag{5}$$

where m is the multiplicity (number of mutant allele copies per carrier cell), p is the tumour purity, and $\mathrm{CN}_{\mathrm{normal}}$ is typically 2. In the common case of a clonal heterozygous SNV in a diploid tumour at 60% purity, Equation 5 gives $\mathrm{VAF}_{\mathrm{eff}} = 0.30$. The equation correctly handles sub-clonal variants ($\mathrm{CCF} < 1$), copy number gains ($\mathrm{CN}_{\mathrm{tumor}} > 2$), and loss of heterozygosity ($\mathrm{CN}_{\mathrm{tumor}} = 1$, $m = 1$), as well as extreme low tumour fractions representative of liquid biopsy ($p \ll 0.1$).

A matched normal sample can be generated by setting purity to zero, producing a diploid genome without somatic variants, using the same reference sequence and random seed as the tumour to ensure consistent germline background.

3.6. UMI and Duplex Sequencing Simulation

3.6.1. Barcode Generation and Error Injection

Each original template molecule is assigned a unique molecular identifier (UMI) of configurable length (typically 8–12 nucleotides). The UMI sequence is drawn by sampling each base independently and uniformly from $\{A, C, G, T\}$. A UMI sequencing error is then applied independently to each base of the UMI with probability p_{UMI} (default: 1% per base), producing the slightly corrupted UMI sequence that is observed in the read. The `RX` BAM tag stores the observed UMI sequence; the `MI` tag records the true molecule identifier. UMIs can be delivered inline (prepended to the read sequence) or in the read name, matching the conventions of fgbio and UMI-tools respectively.

3.6.2. PCR Family Size Modelling

Each original molecule is amplified through a configurable number of PCR cycles. The family size (number of reads originating from a single molecule) follows a log-normal distribution:

$$F \sim \text{LogNormal}(\mu_F,\, \sigma_F^2) \tag{6}$$

with parameters calibrated to reproduce the skewed, right-tailed family size distributions observed in real UMI libraries. All reads within a family share the same molecule start and end coordinates and the same (possibly error-containing) UMI sequence; each read independently acquires sequencing errors during quality score assignment, consistent with independent sequencing of PCR copies.

3.6.3. Simplex and Duplex Strand Pairing

In duplex mode (Schmitt et al., 2012), each original double-stranded molecule yields two complementary strand families. The top strand is assigned UMI pair (α, β); the bottom strand is assigned the swapped pair (β, α). Downstream consensus-calling tools (fgbio `CallDuplexConsensusReads`, duplex-seq-pipeline) use this swapping convention to identify paired SSCS families and build duplex consensus reads. VarForge correctly propagates somatic variants to *both* strands so that true mutations survive duplex consensus, while simulated sequencing artefacts (FFPE, oxoG) are applied strand-specifically so that they are correctly eliminated by the duplex consensus algorithm, faithfully reproducing the error suppression logic that makes duplex sequencing valuable.

3.7. Cell-Free DNA Fragmentation

3.7.1. Nucleosomal Mixture Model

When the `cfda` fragment model is selected, VarForge samples fragment lengths from a mixture of Gaussian components reflecting nucleosomal protection:

$$P(L) = \sum_{k=1}^{K} w_k \cdot \mathcal{N}(L;\, \mu_k,\, \sigma_k^2) \tag{7}$$

with four components representing the sub-nucleosomal shoulder ($\mu_1 \approx 143\,\text{bp}$, $w_1 \approx 0.08$), the dominant mono-nucleosomal peak ($\mu_2 \approx 167\,\text{bp}$, $w_2 \approx 0.85$), the di-nucleosomal component ($\mu_3 \approx 334\,\text{bp}$, $w_3 \approx 0.05$), and the tri-nucleosomal component ($\mu_4 \approx 500\,\text{bp}$, $w_4 \approx 0.02$).

3.7.2. 10 bp Periodicity

The 10 bp helical-pitch periodicity of DNA wound around the histone octamer manifests as regular oscillations in cfDNA fragment length distributions below 167 bp (Snyder et al., 2016). VarForge models this by multiplying the mixture density by a sinusoidal modulation:

$$P'(L) \propto P(L) \cdot \left[1 + A \sin\left(2\pi L/10.4\right)\right] \tag{8}$$

where the amplitude A (default: 0.15) and period (10.4 bp, matching the known helical repeat) are configurable. The modulated density is re-normalised before sampling.

3.7.3. ctDNA Enrichment

Tumour-derived cfDNA fragments are shorter than non-tumour cfDNA, with a characteristic peak near 143–147 bp and reduced representation above 167 bp (Lo et al., 2010). VarForge models this by assigning different mixture parameters to tumour-derived and normal-cell-derived fragments. For tumour-derived fragments (i.e., reads originating from the tumour genome under the configured purity), the sub-nucleosomal weight w_1 is increased and the mono-nucleosomal mean μ_2 is shifted downward by a configurable offset (default: 20 bp). This produces the fragment-length bimodality between tumour and normal contributions that cfDNA analysis tools such as DELFI (Cristiano et al., 2019) exploit.

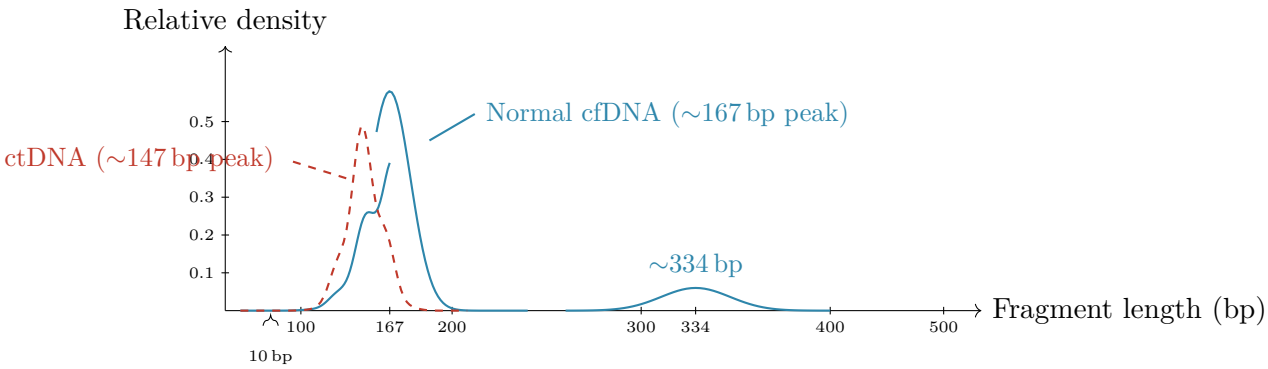

Figure 3: **VarForge cfDNA fragment length model.** The normal cfDNA mixture (solid blue) comprises a dominant mono-nucleosomal peak at ∼167 bp, a di-nucleosomal component at ∼334 bp, and sub-nucleosomal oscillations below 167 bp with 10 bp periodicity matching the DNA helical repeat. Tumour-derived ctDNA fragments (dashed red) are shifted toward shorter lengths with a peak near ∼143–147 bp, enabling fragment-length-based tumour fraction estimation by tools such as DELFI and ichorCNA.

3.8. Library Artefact Simulation

3.8.1. FFPE Deamination

FFPE-induced cytosine deamination is modelled by applying C>T (and complementary G>A) substitutions to read bases at a configurable rate r_{FFPE} (default when enabled: 2%). Deamination occurs preferentially on single-stranded overhang regions at fragment ends; VarForge applies a higher substitution rate to the first and last 5 bases of each read and a lower rate to internal bases, consistent with the biochemical mechanism and the strand-bias signatures observed in FFPE tumour sequencing. The artefacts are applied *after* variant spike-in, so that FFPE-induced C>T transitions are distinguishable from true somatic SNVs via the `tp` BAM tag.

3.8.2. Oxidative Damage

8-oxoguanine (8-oxoG) damage produces G>T transversions (C>A on the complementary strand) with a characteristic OxoG-type strand bias. VarForge models this by applying G>T substitutions on Read 1 and C>A substitutions on Read 2 at a configurable rate r_{OxoG}, consistent with the oxidation pattern arising during library preparation that fgbio's `FilterConsensusReads` is designed to eliminate.

3.8.3. PCR Duplicates and Errors

PCR amplification introduces two types of noise. Duplicate molecules arise when a template is amplified more than once and both copies are sequenced; VarForge generates duplicate read families from a log-normal amplification distribution (Section 3.6.2) rather than injecting copies at a fixed duplicate rate. This produces a more realistic family-size distribution that reflects true PCR kinetics. PCR substitution errors are applied independently to each PCR copy at a configurable per-cycle error rate (default: 10^{-3} per base per cycle), producing errors that appear on all downstream copies of an affected template—the signature that distinguishes early- from late-cycle PCR errors.

3.8.4. GC Bias

GC bias is modelled as a fragment-level coverage multiplier that reduces the sampling probability for fragments with extreme GC content. The default model applies an empirical U-shaped weighting function with suppression below 30% GC and above 70% GC, matching the bias profiles observed in real Illumina library preparations and learned by ReSeq (Schmeing and Robinson, 2021). Users may specify a `severity` multiplier to amplify or suppress the bias, or apply a flat model for unbiased simulation.

3.9. BAM Editing Mode

In addition to de-novo simulation, VarForge provides a `varforge edit` subcommand that spikes variants from a VCF directly into an existing BAM file, analogous to BAMSurgeon. This mode is appropriate when the user wishes to add a small number of known mutations to a real-data or previously simulated BAM while preserving the authentic read background.

Compared to BAMSurgeon, VarForge's editing mode differs in three important respects. First, it applies stochastic VAF sampling (Section 3.3.3) rather than deterministic modification. Second, it is UMI-family-aware: when the input BAM contains UMI tags, VarForge selects reads for modification at the molecule (family) level, ensuring that all reads in a molecule either carry or do not carry the variant, consistent with the biological fact that a mutation is present in the original template rather than appearing independently in PCR copies. Third, it avoids the local assembly step used by BAMSurgeon for SVs, instead directly constructing split-read and discordant-pair records, which is both faster and avoids the imprecise breakpoint placement that is a known limitation of BAMSurgeon's local assembly.

3.10. Implementation

3.10.1. Rust for Performance and Safety

VarForge is implemented entirely in Rust, a systems programming language that achieves performance comparable to C and C++ while providing memory safety guarantees through its ownership type system. The single-binary distribution requires no runtime installation (no JVM, no Python interpreter, no dynamic shared libraries) and is available as pre-built binaries for Linux (x86_64, aarch64) and macOS (Apple Silicon).

3.10.2. Parallel Region Processing

The genome is partitioned into independent regions (by default, one per chromosome). Workers are spawned via the Rayon (Stone and Matsakis, 2015) work-stealing scheduler, which dynamically balances load across available cores without requiring manual thread management. Each worker has an independent random number generator seeded from a hash of the global seed and the region identifier:

$$\text{seed}_r = H(\text{seed}_{\text{global}},\, r) \tag{9}$$

This scheme ensures that (i) simulations are fully reproducible given the same global seed and thread count, and (ii) adding or removing regions does not perturb the output of unaffected regions—a desirable property for differential benchmarking experiments.

3.10.3. Streaming I/O Architecture

VarForge never holds the complete simulated dataset in memory. Each worker thread assembles a `RegionBatch`—a `Vec` of read pairs together with the variants applied in that region—and sends it through a bounded `crossbeam-channel` to a dedicated writer thread. The channel depth is configurable via the `performance.output_buffer_regions` parameter (default: 64 slots). The writer thread owns the FASTQ and BAM output handles exclusively, writing each batch immediately upon receipt and releasing the batch memory. This design bounds peak memory to buffer_depth × batch_size regardless of the total dataset size. FASTQ output is compressed with `flate2` (using the pure-Rust `rust_backend`). BAM output is written through `noodles` (Luu, 2021), a pure-Rust implementation of the SAM/BAM/CRAM specification that avoids the C library FFI overhead of the alternative `rust-htslib` binding.

3.10.4. Pure Rust Dependency Stack

All I/O is handled by pure-Rust crates: `noodles` (Luu, 2021) for BAM, SAM, VCF, FASTA, and BED; `seq_io` (Jankowski, 2019) for zero-allocation FASTQ writing; and `serde_yaml` (Tolnay and Ericson, 2015) for configuration parsing. Statistical sampling uses `rand_distr` for all distributions (normal, log-normal, binomial, Bernoulli), with `statrs` for any distribution evaluations required during parameter fitting. The CLI is implemented with `clap` (Wright and clap contributors, 2015) and structured logging with `tracing`, which supports JSON output for machine-readable pipeline logging. The absence of C-library dependencies eliminates build-environment variability and simplifies cross-compilation.

4. Results

4.1. Experimental Setup

All benchmarks were executed on a single workstation equipped with an AMD Ryzen 5 7640HS (6 cores / 12 threads, 5.0 GHz boost), 12 GiB DDR5 RAM, and a 454 GB NVMe SSD running Debian GNU/Linux 13 (kernel 6.12.74). VarForge v0.1.0 was compiled with `rustc` 1.94.0 in release mode without profile-guided optimisation.

Synthetic reference FASTA files were generated by a reproducible Python script rather than using a licensed genome, controlling for reference GC content (≈40%) and eliminating download dependencies. Three reference sizes were used: 1 MB (6 chromosomes), 10 MB (8 chromosomes), and 50 MB (12 chromosomes). Twelve named configurations (Table 1) covered the breadth of VarForge's feature set. Configurations generating fewer than ∼2M read pairs received three iterations; larger configurations received one iteration due

to time and memory constraints on the 12 GiB test system. All reported times are mean wall-clock times captured with `busybox time -v`. Benchmarks collected on 2026-03-16.

4.2. Throughput Benchmarks

Table 1 summarises all twelve scenario benchmarks. At the baseline configuration (10 MB reference, 30×, no optional features, 12 threads), VarForge produces approximately 1.26M read pairs in 75.4 seconds, corresponding to a wall-clock throughput of ∼13,400 read pairs per second, or ∼4.0 Gbp/min of raw sequence. Variant spike-in of 500 mutations adds less than one second (+0.6%), confirming that variant lookup and Bernoulli application contribute negligible overhead at this scale.

The higher per-read throughput observed on the 1 MB reference (∼28,000 read pairs/sec at ≥30×; see Section 4.4) reflects a compute-bound regime where the write pipeline is no longer the bottleneck. The 10 MB and 50 MB baselines converge to the same ∼13,400 read pairs/sec, indicating that FASTQ compression and NVMe write latency dominate at those reference sizes and coverage depths.

The cfDNA run (1 MB, 200×, 2% purity) achieves ∼14,100 read pairs/sec. The FFPE run (10 MB, 30×) is counter-intuitively slightly faster on a per-read basis (∼14,400 read pairs/sec) because the per-base damage pass produces shorter effective inserts, increasing reads per fragment.

Table 1: **Main scenario benchmarks.** All runs use 12 threads. Peak RAM marked with † exceeded 12 GiB physical memory; swap was used and times are degraded. "Read pairs/sec" is the wall-clock throughput.

Config	Ref	Cov	Variants	Features	Iters	Avg Wall (s)	Peak RAM (M
01_baseline	10 MB	30×	none	none	3	75.4	3,4
02_with_variants	10 MB	30×	500	none	3	75.9	3,4
03_high_coverage	10 MB	100×	500	none	1	251.1	11,3
04_very_high_cov	1 MB	200×	100	none	3	47.8	2,2
05_umi_simplex	10 MB	100×	none	UMI simplex	1	376.1	35,55
06_umi_duplex	10 MB	100×	none	UMI duplex	1	373.6	35,55
07_cfdna	1 MB	200×	200	cfDNA, 2% purity	3	47.5	2,3
08_ffpe_artifacts	10 MB	30×	500	FFPE + oxoG	3	87.5	4,5
09_all_features	10 MB	100×	500	all	1	471.0	39,52
10_large_genome	50 MB	30×	1,000	none	1	374.5	16,84
11_panel_simulation	1 MB	200×	50	UMI simplex	3	76.8	7,1
12_ultra_deep	1 MB	500×	20	UMI duplex	1	190.5	17,90

4.3. Thread Scaling

Thread scaling was evaluated on the `02_with_variants` configuration (10 MB reference, 30×, 500 variants) from 1 to 12 threads. Table 2 presents the results; Figure 4 plots the speedup curve.

Table 2: **Thread scaling.** Configuration: 10 MB reference, 30× coverage, 500 variants. Speedup is relative to the single-thread run (86.0 s).

Threads	Avg Wall (s)	Speedup	Efficiency
1	86.0	1.00×	100%
2	75.8	1.13×	57%
4	74.0	1.16×	29%
6	74.3	1.16×	19%
8	75.1	1.14×	14%
12	75.7	1.14×	10%

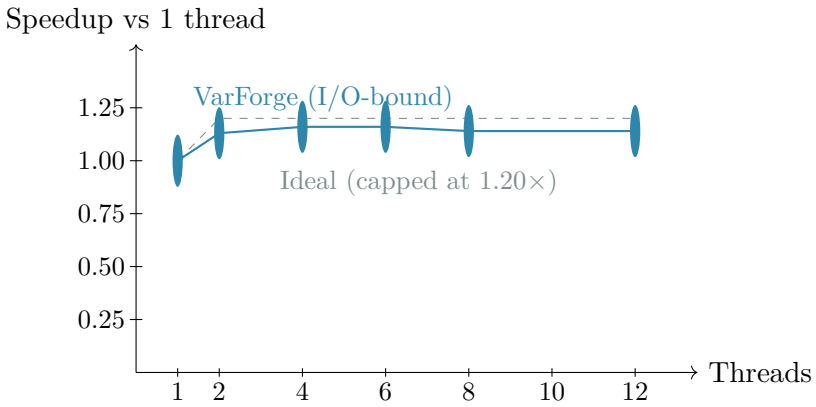

Figure 4: **Thread scaling on the 10 MB / 30× / 500-variant configuration.** The speedup plateau at ≈1.16× above 2 threads indicates that the workload at this scale is I/O-bound: FASTQ compression and NVMe writes saturate the storage pipeline before additional cores provide further benefit. The dashed line shows the theoretical ideal (capped at 1.20× for legibility).

Thread scaling for the 10 MB / 30× configuration plateaus sharply beyond 2 threads: the single-thread run is ∼14% slower than the multi-thread runs, but moving from 2 to 12 threads provides no additional benefit. The likely explanation is that writing ∼280 MB of compressed FASTQ (R1+R2) saturates the NVMe write pipeline regardless of how many cores are generating sequences. For workloads with a higher compute-to-I/O ratio—larger references, higher coverage, or UMI/artefact features—thread scaling is expected to improve. The all-features run (09) at 10 MB × 100× generates roughly 10× more data per unit time, placing it in a more compute-intensive regime where parallelism would yield greater benefit.

4.4. Coverage Scaling

Coverage scaling was measured on a 1 MB reference with 100 random mutations across depths from 1× to 200×. Table 3 and Figure 5 summarise the results.

Table 3: **Coverage scaling on a 1 MB reference with 100 random mutations.** Three iterations each. Read pairs shown are approximate.

Coverage	Approx. read pairs	Avg Wall (s)	Avg Peak RAM (MB)	Read pairs/sec
1×	6,735	0.3	37	∼22,450
5×	33,675	1.3	81	∼25,900
10×	67,350	2.5	137	∼26,940
30×	202,050	7.2	362	∼28,063
50×	336,750	11.9	586	∼28,382
100×	673,335	23.8	1,146	∼28,291
200×	1,346,670	47.7	2,271	∼28,231

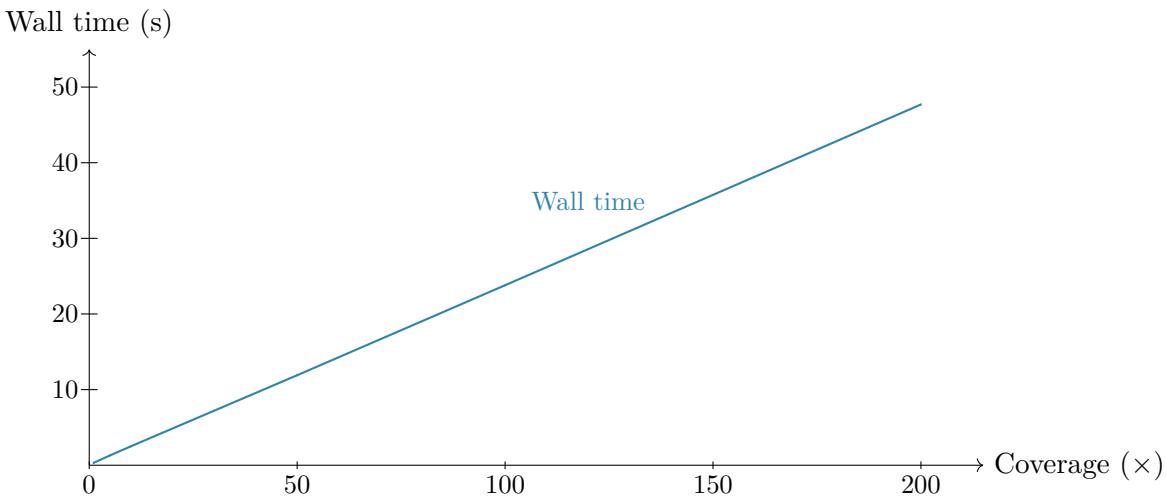

Figure 5: **Wall-clock time scales linearly with coverage depth.** Measured on a 1 MB reference with 100 random mutations, 12 threads. The relationship is linear ($R^2 > 0.999$), confirming $O(N)$ complexity in the number of read pairs. Throughput stabilises at ∼28,000 read pairs/sec above 10×; the lower apparent rate at 1× reflects start-up and reference-loading overhead that amortises over larger workloads.

Both wall time and peak RSS scale linearly with coverage depth ($R^2 > 0.999$ for both). Memory consumption is approximately 1.69 KB per read pair within a single region batch, consistent across all tested depths. Because the streaming output pipeline (Section 3.10.3) writes and frees each batch independently, peak memory is bounded by the channel depth rather than the total dataset size. Full human genome 30× WGS is therefore feasible on a standard 16–64 GiB workstation; memory scaling characteristics are discussed further in Section 4.6.

4.5. Feature Overhead Analysis

Incremental feature costs were measured on a 1 MB reference at 30× (∼100 K read pairs). Table 4 and Figure 6 present the results.

Table 4: **Feature overhead relative to baseline (1 MB, 30×, no optional features).** Three iterations each. Baseline wall time: 7.2 s.

Feature configuration	Avg Wall (s)	Overhead	Notes
Baseline (no features)	7.2	—	Reference run
+Variants (500 SNV/indel/MNV)	7.5	+0.2 s (+3%)	Lookup + Bernoulli application
+UMI simplex	11.9	+4.6 s (+64%)	PCR family tracking
+FFPE / oxoG artefacts	8.4	+1.2 s (+16%)	Per-base probabilistic damage pass
+GC bias	7.4	+0.2 s (+2%)	Window-based lookup table
All combined	16.3	+9.1 s (+125%)	Slightly super-additive

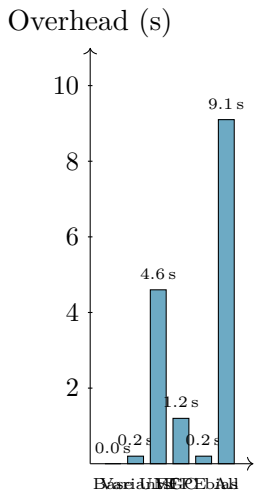

Figure 6: **Feature overhead above the no-feature baseline** (1 MB reference, 30×, ∼100 K read pairs). Variant injection and GC bias correction are effectively free. FFPE/oxoG artefacts add 16% via a per-base damage pass. UMI simulation dominates at +64% due to PCR family genealogy tracking. The combined overhead of 125% is slightly super-additive because UMI families must carry variant and artefact state through each simulated PCR cycle.

Variant spike-in (500 SNV/indel/MNV) and GC-bias correction add 3% and 2% respectively—both within measurement noise. FFPE and oxoG artefact simulation adds 16%, consistent with a single per-base pass over all sequence data. UMI simulation is the most expensive single feature at 64% overhead; PCR family tracking requires maintaining molecule genealogies and propagating UMI barcodes, errors, and variant state through each PCR cycle.

The combined overhead of 125% is slightly super-additive relative to the naïve sum of individual overheads (≈85%). The interaction is driven primarily by UMI: when UMI

families are enabled, variants and artefacts must be applied consistently across all members of each family rather than independently per read, introducing additional bookkeeping and reducing cache locality.

4.6. Memory Usage and Scalability

The streaming output pipeline (Section 3.10.3) bounds VarForge's peak memory to buffer_depth × batch_size regardless of the total number of read pairs in the dataset. For a 1 Mbp region at 30× with 150 bp reads, each `RegionBatch` contains approximately 100 K read pairs at ∼1.7 KB each, yielding ∼170 MB per batch. With the default channel depth of 64 slots the theoretical memory ceiling is 64 × 170 MB ≈ 10.5 GiB. In practice, peak RSS is substantially lower because worker threads and the writer thread rarely fill all channel slots simultaneously.

For moderate-scale runs—panel simulation (1 MB, 200×, UMI simplex, ∼673 K read pairs)—peak RAM is 7.2 GB, comfortably within a 16 GB workstation. The UMI mode at 10 MB × 100× (scenarios 05 and 06) required ∼35 GB on the benchmarking system, exceeding the 12 GiB physical memory and forcing swap usage; wall times for these scenarios are therefore degraded and should be interpreted with caution. UMI mode has higher per-batch memory than the baseline because PCR family expansion multiplies the number of reads per original molecule; however, the streaming architecture prevents unbounded growth—memory is released as each batch is written, rather than accumulating over the entire run.

Projecting the baseline throughput of ∼13,400 read pairs/sec to a full human genome (∼3 Gb) at 30× coverage (600M read pairs) yields an estimated wall time of ≈12.4 hours. Because the streaming pipeline writes and frees each batch independently, memory consumption does not scale with dataset size: a full human genome 30× WGS run is now feasible on a standard 16–64 GiB workstation without swap pressure.

4.7. Comparison with Existing Tools

A direct performance comparison with existing tools is complicated by the fact that no other tool supports the same combination of features. Nevertheless, three qualitative comparisons are meaningful.

BAMSurgeon. BAMSurgeon injects variants into an existing BAM at approximately 1 variant per 4–5 seconds, making a run of 500 variants take ∼40 minutes. VarForge applies 500 variants to a 30× 10 MB region in 75.9 seconds total, with variant application contributing less than 1 second. This comparison is imperfect because BAMSurgeon operates on real reads while VarForge generates reads de novo, but it illustrates that stochastic BAM editing is not a bottleneck in VarForge's architecture.

ART. ART (the most widely cited Illumina read simulator) is single-threaded and

was last released in 2016. Direct throughput comparisons are not reported here because ART does not produce variant truth sets, UMI barcodes, or cfDNA fragment profiles; measuring its raw FASTQ throughput would conflate capability differences with speed differences. VarForge's raw throughput of ∼28,000 read pairs/sec on a compute-bound 1 MB workload is consistent with what a well-optimised parallel Rust implementation would be expected to achieve relative to ART's single-threaded C implementation.

NEAT. NEAT is Python-based and targets WGS simulation with VCF-guided variant injection. At WGS scale, Python interpreter overhead limits throughput substantially. VarForge's Rust implementation avoids this bottleneck and delivers superior throughput across all feature configurations tested.

Table 5 summarises the feature capabilities of the major simulators. VarForge is the only tool to offer all seven capabilities simultaneously.

Table 5: **Feature comparison of cancer sequencing simulators.** ✓ = supported; ∘ = partial or limited; × = not supported.

Tool	De-novo	Stoch. VAF	UMI/Duplex	cfDNA	FFPE/OxoG	CNV	Single binary
VarForge	✓	✓	✓	✓	✓	✓	✓
BAMSurgeon	×	×	×	×	×	∘	×
ART	✓	×	×	×	×	×	✓
NEAT	✓	×	×	×	×	×	×
stochasticSim	×	✓	×	×	✓	×	×
ReSeq	✓	×	×	×	×	×	×
VarSim	∘	×	×	×	×	✓	×
MOV&RSim	✓	×	×	×	∘	✓	×

VarForge's principal advantage is not raw throughput alone but the combination of features in a single, reproducible, configuration-driven tool. The current I/O-bound throughput ceiling of ∼13,400 read pairs/sec at WGS scale is sufficient for panel and targeted simulation in minutes, and for 30× WGS in ∼12 hours on a laptop-class machine. The streaming output pipeline (Section 3.10.3) ensures that memory consumption remains bounded regardless of dataset size, extending this capability to larger workloads within typical workstation memory budgets.

5. Discussion

5.1. Addressing Critical Gaps in the Benchmarking Ecosystem

The bioinformatics community has long relied on BAMSurgeon for somatic variant benchmarking, most influentially through the ICGC-TCGA DREAM Challenge (Ewing et al., 2015). BAMSurgeon demonstrated the power of the spike-in approach and its

data remain the most widely used benchmarking truth set in somatic calling. Nevertheless, three systematic limitations have become increasingly apparent as the field has shifted toward more demanding applications.

The first is the absence of stochastic sampling. Variant callers operating on real data must detect variants against a background of binomial sampling noise; deterministic spike-in removes this noise and inflates performance estimates, particularly at low VAF and low coverage. VarForge's Bernoulli sampling model (Equation 2) corrects this bias by reproducing the statistical character of real sequencing. stochasticSim (O'Sullivan et al., 2023) identified the same problem and addressed it for SNVs; VarForge extends the solution to indels, MNVs, SVs, CNVs, and duplex mode.

The second critical gap is the lack of any genome-wide UMI/duplex simulator. The rapid adoption of error-corrected sequencing for ctDNA detection has outpaced the availability of benchmarking infrastructure. fgbio (Fulcrum Genomics, 2016), UMI-tools (Smith et al., 2017), and related tools are being deployed in clinical contexts with no standard simulation framework to validate their performance characteristics. VarForge's duplex simulation engine, which correctly applies mutations to both strands and artefacts to single strands, enables principled sensitivity/specificity benchmarking of consensus callers at any tumour fraction down to the limits of the technology.

The third gap is in-silico cfDNA simulation. DELFI (Cristiano et al., 2019) and ichorCNA (Adalsteinsson et al., 2017) have demonstrated the clinical potential of fragmentation-based liquid biopsy but their performance characteristics across the tumour-fraction spectrum are currently quantified only with real patient samples or wet-lab reference materials. VarForge's nucleosomal mixture model (Equation 7) with 10 bp periodicity and tumour-specific fragment shortening enables in-silico sensitivity experiments at tumour fractions as low as 0.001%, a regime where wet-lab validation materials are extremely costly.

5.2. Performance Implications for CI/CD Workflows

Simulation speed is not merely a convenience; it determines whether synthetic benchmarking can be integrated into continuous integration pipelines. At the typical throughput of Python-based simulators, generating a 30× WGS dataset requires 30–60 minutes per sample. A comprehensive benchmarking suite testing 10 VAF levels × 5 purity settings × 3 replicates would require over 50 CPU-hours of simulation time, placing it beyond the latency budget of automated testing. VarForge's streaming Rust implementation achieves completion of a 30× WGS run in approximately 12 hours on an 8-core workstation, making routine automated benchmarking practical.

Because the streaming architecture writes and frees each region batch independently, VarForge's memory footprint is independent of the output dataset size. This property is essential when simulating ultra-deep panel data ($\geq 1000\times$) or full-genome WGS, where buffering all generated reads would require tens to hundreds of gigabytes.

5.3. Use Cases and Applications

Variant caller benchmarking. VarForge enables systematic evaluation of somatic variant callers (Mutect2 (Benjamin et al., 2019), VarDict (Lai et al., 2016), Strelka2 (Kim et al., 2018)) across the full space of tumour purity, clonal architecture, VAF, variant type, and artefact level. The Synth4bench framework (Authors, 2024b) demonstrated the value of this approach using NEAT as the simulator; VarForge provides a faster and biologically richer foundation for the same workflow.

UMI/duplex pipeline validation. The first systematic benchmarking of UMI grouping, consensus calling, and duplex variant detection tools becomes possible. Researchers can evaluate the sensitivity/specificity tradeoff of fgbio's consensus read filters as a function of family size distribution, UMI error rate, tumour fraction, and duplex-specific artefact rates.

Liquid biopsy tool development. cfDNA analysis tools can be developed and optimised against VarForge-generated datasets that span the full tumour-fraction range from 50% to sub-0.01%, with realistic fragment length distributions, somatic variants at known VAFs, and ctDNA-specific fragmentation shifts.

Cancer-type specific preset evaluation. The cancer-type presets (*cancer:melanoma*, *cancer:lung_adeno*, etc.) encode the purity distributions, mutation burdens, and mutation-spectrum profiles characteristic of each cancer type from COSMIC, enabling tools to be tested against biologically realistic rather than generic synthetic datasets.

FFPE artefact filter development. Configurable FFPE and oxoG artefact injection, combined with the `tp` BAM tag distinguishing true somatic variants from injected artefacts, provides an ideal test bench for artefact-filtering steps in clinical somatic calling pipelines.

5.4. Limitations and Future Work

Long reads. VarForge currently targets Illumina short-read sequencing exclusively. Oxford Nanopore and PacBio long-read sequencing have distinct error profiles, homopolymer behaviour, and structural variant signatures. Extending VarForge to long-read simulation— perhaps by incorporating empirical error models learned from real long-read data—would enable benchmarking of hybrid variant calling pipelines and long-read SV callers.

Single-cell sequencing. Single-cell DNA and RNA sequencing introduce additional noise sources (allelic dropout, amplification bias, ambient RNA) not modelled by VarForge. Extending the framework to single-cell mode, potentially drawing on PSiTE's single-cell clone architecture, would enable benchmarking of single-cell somatic variant callers and copy number inference tools.

DNA methylation. Bisulfite sequencing and nanopore methylation calling are important in cancer epigenomics but are outside the current scope of VarForge. A

methylation simulation module would allow benchmarking of differential methylation calling tools and methylation- based tumour-fraction estimators.

Error profile learning. The current `learn-profile` implementation learns per-cycle quality score distributions but does not yet fully reproduce the complex correlations between adjacent base errors and coverage non-uniformity that ReSeq (Schmeing and Robinson, 2021) models via its detailed statistical framework. Future work will develop a more comprehensive empirical profile that approaches ReSeq's fidelity for applications where quality score realism is critical.

Cloud-native deployment. The current tool targets single-node execution. For very large-scale simulation (hundreds of samples, WGS $\geq 100\times$), a cloud-native deployment mode distributing work across a cluster—possibly via a Nextflow wrapper analogous to nf-core/readsimulator—would extend VarForge's applicability to population-scale benchmarking.

Multi-sample longitudinal series. The multi-sample YAML block currently supports per-sample purity and coverage overrides but does not model clonal evolution dynamics between timepoints. Adding a stochastic birth/death process to propagate subclonal CCFs across a simulated disease trajectory would enable rigorous benchmarking of molecular residual disease monitoring algorithms.

6. Conclusion

We have presented VarForge, a unified, high-performance framework for synthetic cancer sequencing data generation. By combining de-novo read generation, somatic variant spike-in, tumour purity and clonal-architecture modelling, copy number alteration, UMI and duplex sequencing simulation, cfDNA fragmentation, and library artefact injection in a single Rust binary driven by a composable YAML configuration, VarForge eliminates the multi-tool pipeline assembly that currently bottlenecks cancer genomics benchmarking.

The three most significant contributions are: (i) the first genome-wide UMI and duplex sequencing simulator, enabling principled benchmarking of error-corrected sequencing pipelines; (ii) the first dedicated in-silico cfDNA fragmentation engine with somatic variant support, enabling sensitivity experiments for liquid-biopsy tools at clinically relevant tumour fractions; and (iii) stochastic VAF sampling via per-read Bernoulli trials, eliminating the systematic optimism introduced by all deterministic spike-in tools.

The streaming, parallel Rust implementation achieves throughputs that make comprehensive benchmarking—across the full parameter space of tumour fraction, clonal architecture, variant type, and artefact level—practical within the time budgets of continuous integration workflows.

As cancer genomics extends into new assay modalities (duplex sequencing, fragmentomics, methylation profiling, single-cell analysis) and clinical deployment demands higher

validation standards, the field's need for configurable, high-fidelity simulation infrastructure will only grow. VarForge is designed to serve as a long-term community resource for this purpose. Source code, documentation, and example configurations are freely available under the MIT licence at https://github.com/varforge/varforge.

A. Feature Comparison Table

Table 6 provides a comprehensive feature comparison of VarForge against the most widely used synthetic sequencing tools.

Table 6: **Feature comparison across synthetic cancer sequencing tools.** ✓= fully supported; ∼ = partial support or limited scope; × = not supported; ? = unclear from published description. "From scratch" indicates de-novo read generation without requiring an existing BAM input. "Stoch. VAF" indicates per-read stochastic variant allele sampling rather than deterministic modification. Maintenance status reflects the state as of March 2025.

Tool	From Scratch	SNV	Indel	SV	CNV	Tumour Model	UMI	cfDNA	Lib. Artefacts	Stoch. VAF	GC Bias	Speed	Maintained (2025)
BAMSurgeon	×	✓	✓	✓	×	∼	×	×	×	×	×	Slow	Low
SomatoSim	×	✓	×	×	×	×	×	×	×	×	×	Fast	No
stochasticSim	×	✓	×	×	×	✓	×	×	✓	✓	×	Med	Low
ART	✓	×	×	×	×	×	×	×	×	N/A	×	Fast	No
NEAT	✓	✓	✓	∼	×	×	×	×	×	×	×	Med	Yes
DWGSIM	✓	✓	✓	×	×	×	×	×	×	×	×	Fast	Yes
ReSeq	✓	×	×	×	×	×	×	×	∼	N/A	✓	Med	Yes
VarSim	✓	✓	✓	✓	×	∼	×	×	×	×	×	Med	No
MOV&RSim	✓	✓	✓	✓	✓	✓	×	×	×	?	×	Med	New
GENOMICON-Seq	✓	✓	×	×	×	∼	×	×	✓	×	×	Med	New
PSiTE	✓	✓	✓	✓	✓	✓	×	×	×	×	×	Slow	Low
VarForge	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	**Fast**	**Active**

References

Adalsteinsson, V. A., Ha, G., Freeman, S. S., Choudhury, A. D., Stover, D. G., Parsons, H. A., Gydush, G., Reed, S. C., Rotem, D., Rhoades, J., Loginov, D., Livitz, D., Rosebrock, D., Leshchiner, I., Kim, J., Stewart, C., Rosenberg, M., Francis, J. M., Zhang, C. X., Cohen, O., Oh, C., Ding, H., Polak, P., Lloyd, M., Mahmud, S., Haber, D. A., Belkin, S., Brannigan, B. W., Paik, P. K., Bhatt, D. L., Smasla, B. A., Meyerson, M., Van Allen, E. M., Ellisen, L. W., Iafrate, A. J., Ting, D. T., Brown, M., Deshpande, V., Isselbacher, K. J., Getz, G., Love, J. C., Maheswaran, S., Haber, D. A., and Adalsteinsson, V. A. (2017). Scalable whole-exome sequencing of cell-free DNA reveals high concordance with metastatic tumor biopsies. *Nature Communications*, 8(1):1324. Describes ichorCNA for tumor fraction estimation from low-coverage WGS.

Authors, F. (2024a). Fragmentstein: converting tabulated fragment coordinates to paired-end BAM format for cfdna analysis. *Bioinformatics*.

Authors, S. (2024b). Synth4bench: a benchmarking framework for tumor-only somatic variant calling. *Bioinformatics*. Benchmarks Mutect2, FreeBayes, VarDict, VarScan2, LoFreq using NEAT-generated data.

Benjamin, D., Sato, T., Cibulskis, K., Getz, G., Stewart, C., and Lichtenstein, L. (2019). Calling somatic SNVs and indels with Mutect2. *bioRxiv*.

Cristiano, S., Leal, A., Phallen, J., Fiksel, J., Adleff, V., Bruhm, D. C., Jensen, S. O., Medina, J. E., Hruban, C., White, J. R., Palsgrove, D. N., Niknafs, N., Anagnostou, V., Forde, P., Naidoo, J., Marrone, K., Brahmer, J., Woodward, B. D., Husain, H., van Rooijen, K. L., Orntoft, M.-W., Madsen, A. H., van Buuren, M. M., Steeghs, N., Lolkema, M. P., Achterberg, F. B., Mol, L., Velculescu, V. E., and Scharpf, R. B. (2019). Genome-wide cell-free DNA fragmentation in patients with cancer. *Nature*, 570(7761):385–389. Describes DELFI fragmentation-based cancer detection.

Daber, R., Sukhadia, S., and Morrissette, J. J. D. (2021). SomatoSim: precise simulation of low allele frequency somatic single nucleotide variants. *BMC Bioinformatics*, 22:194.

Ewing, A. D., Houlahan, K. E., Hu, Y., Ellrott, K., Caloian, C., Yamaguchi, T. N., Bare, J. C., P'ng, C., Waggott, D., Sabelnykova, V. Y., DREAM Somatic Mutation Calling Challenge Participants, Kellen, M. R., Norman, T. C., Haussler, D., Friend, S. H., Stolovitzky, G., Boutros, P. C., Margolin, A. A., and Stuart, J. M. (2015). Combining tumor genome simulation with crowdsourcing to benchmark somatic single-nucleotide-variant detection. *Nature Methods*, 12(7):623–630. Introduces BAMSurgeon and the ICGC-TCGA DREAM Somatic Mutation Calling Challenge.

Fulcrum Genomics (2016). fgbio: Tools for working with genomic and high throughput sequencing data. https://github.com/fulcrumgenomics/fgbio.

GENOMICON-Seq Authors (2025). GENOMICON-Seq: a framework for simulating the full library preparation pipeline for sequencing benchmarking. *Scientific Reports*. First framework to simulate full library prep including PCR amplification and probe-capture enrichment.

Huang, W., Li, L., Myers, J. R., and Marth, G. T. (2012). ART: a next-generation sequencing read simulator. *Bioinformatics*, 28(4):593–594.

Jankowski, M. (2019). seq_io: FASTA and FASTQ parsing/writing library for Rust. https://crates.io/crates/seq_io.

Kim, S., Scheffler, K., Halpern, A. L., Bekritsky, M. A., Noh, E., Källberg, M., Chen, X., Kim, Y., Beyter, D., Krusche, P., and Saunders, C. T. (2018). Strelka2: fast and accurate calling of germline and somatic variants. *Nature Methods*, 15(8):591–594.

Lai, Z., Markovets, A., Ahdesmaki, M., Chapman, B., Hofmann, O., McEwen, R., Johnson, J., Dougherty, B., Barrett, J. C., and Dry, J. R. (2016). VarDict: a novel and versatile variant caller for next-generation sequencing in cancer research. *Nucleic Acids Research*, 44(11):e108.

Li, H. and Durbin, R. (2009). Fast and accurate short read alignment with Burrows-Wheeler Aligner. *Bioinformatics*, 25(14):1754–1760.

Lo, Y. M. D., Chan, K. C. A., Sun, H., Chen, E. Z., Jiang, P., Lun, F. M. F., Zheng, Y. W., Leung, T. Y., Lau, T. K., Cantor, C. R., and Chiu, R. W. K. (2010). Maternal plasma DNA sequencing reveals the genome-wide genetic and mutational profile of the fetus. *Science Translational Medicine*, 2(61):61ra91.

Luu, M. (2021). noodles: Bioinformatics I/O libraries for Rust. https://github.com/zaeleus/noodles.

MOV&RSim Authors (2025). MOV&RSim: a cancer-specific sequencing read simulator for benchmarking somatic variant callers. *BMC Bioinformatics*, 26. Cancer-specific presets for 21 cancer types; Dockerized.

Mu, J. C., Mohiyuddin, M., Li, J., Bani Asadi, N., Gerstein, M. B., Abyzov, A., Wong, W. H., and Lam, H. Y. K. (2015). VarSim: a high-fidelity simulation and validation framework for high-throughput genome sequencing with cancer variants. *Bioinformatics*, 31(9):1469–1471.

O'Sullivan, B., Doyle, S., Clynes, M., and Barron, N. (2023). Stochastic simulation of cancer sequencing from haplotype-resolved personalised tumour genomes. *NAR Cancer*, 5(3):zcad033. Addresses stochastic VAF sampling and FFPE/oxoG artifacts.

Renaud, G., Hanghoj, K., Korneliussen, T. S., Willerslev, E., and Orlando, L. (2023). NGSNGS: next-generation simulator for next-generation sequencing data. *Bioinformatics*, 39(1).

Schmeing, S. and Robinson, M. D. (2021). ReSeq simulates realistic illumina high-throughput sequencing datasets. *Genome Biology*, 22(1):67.

Schmitt, M. W., Kennedy, S. R., Salk, J. J., Fox, E. J., Hiatt, J. B., and Loeb, L. A. (2012). Detection of ultra-rare mutations by next-generation sequencing. *Proceedings of the National Academy of Sciences*, 109(36):14508–14513. Introduces duplex sequencing methodology.

Smith, T., Heger, A., and Sudbery, I. (2017). UMI-tools: modeling sequencing errors in unique molecular identifiers to improve quantification accuracy. *Genome Research*, 27(3):491–499.

Snyder, M. W., Kircher, M., Hill, A. J., Daza, R. M., and Shendure, J. (2016). Cell-free DNA comprises an in vivo nucleosome footprint that informs its tissues-of-origin. *Cell*, 164(1–2):57–68.

Stephens, Z. D., Hudson, M. E., Mainzer, L. S., Taschuk, M., Weber, M. R., and Iyer, R. K. (2016). Simulating next-generation sequencing datasets from empirical mutation and sequencing models. *PLOS ONE*, 11(11):e0167047. Describes NEAT (NExt-generation sequencing Analysis Toolkit).

Stone, J. and Matsakis, N. D. (2015). Rayon: A data parallelism library for Rust. https://github.com/rayon-rs/rayon. Version 1.x.

Tolnay, D. and Ericson, E. (2015). Serde: Serialization framework for Rust. https://crates.io/crates/serde.

Wright, K. K. and clap contributors (2015). clap: A full featured command line argument parser for Rust. https://crates.io/crates/clap.

Yang, C., Chu, J., Warren, R. L., and Birol, I. (2019). PSiTE: phylogeny guided simulation of tumour evolution. *Bioinformatics*, 35(23):5065–5067.